

IF 260

Sistem Operasi

Minggu 7

Dr. Ananda Kusuma
e-mail: ananda.kusuma@lecturer.umn.ac.id

Universitas Multimedia Nusantara
Serpong, Tangerang

Agenda

- Review materi
- Quiz

Review materi:

- Process/Thread**
- Race Condition**
- Sinkronisasi: Semaphores**

Process (Proses)

- **Process adalah**
 - Program yang sedang dieksekusi (berjalan)
 - Container yang berisikan seluruh informasi sumber daya yang diperlukan untuk menjalankan suatu program
 - Program : static file (image), misalnya executable file atau library file
 - Process : executing program = program + execution state
- **Tiap process memiliki PID (Process ID)**
 - Tiap process itu unik (PID berbeda menandakan process yang berbeda)
- **Program yang sama dapat dieksekusi beberapa kali, dan tiap eksekusi diabstrasikan dengan process yang berbeda.**
 - Abstraksi pemrosesan CPU pada beberapa program (multi-tasking) sehingga diperoleh nuansa seolah-olah program-program tersebut berjalan bersamaan (concurrency)

Process Resource Information (Informasi sumber daya proses)

- **Tiap process juga memiliki**
 - **Atribut/informasi pada process table atau process control block (PCB) yang berisikan berbagai sumber daya spt. Informasi registers (program counter, stack pointer), open files, alarm, process-process terkait, dsb.**

Process management	Memory management	File management
Registers Program counter Program status word Stack pointer Process state Priority Scheduling parameters Process ID Parent process Process group Signals Time when process started CPU time used Children's CPU time Time of next alarm	Pointer to text segment info Pointer to data segment info Pointer to stack segment info	Root directory Working directory File descriptors User ID Group ID

Process Creation (Pembuatan Proses)

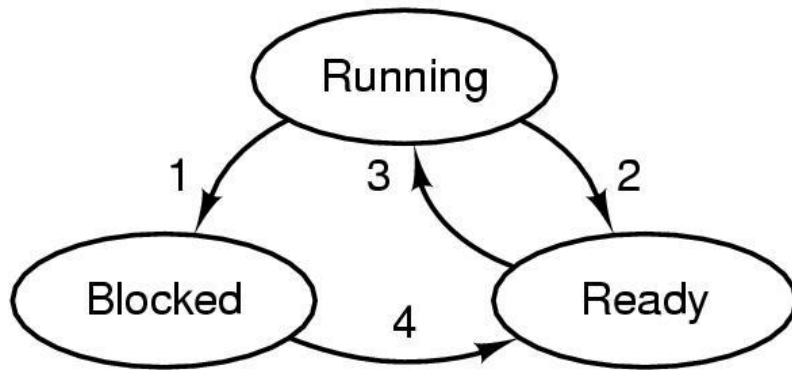
- **Process dapat dibuat dan dihapus pada sistem secara dinamis**
- **Events yang menyebabkan pembuatan process**
 - **Inisialisasi sistem**
 - **Pengguna mengeksekusi command**
 - **Eksekusi system call, contoh: fork()**
 - **Inisiasi batch job**
- **Foreground vs background process**

Contoh: hirarki process dengan system call fork()

```
demo-fork2.c
1 #include <unistd.h>
2 #include <stdio.h>
3
4 int main()
5 {
6     fork();
7     fork();
8     fork();
9     printf("%d %d: Hello!\n",getppid(),getpid());
10 }
```

Process States

- **Process memiliki execution states sbb.:**



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

Ready: siap dieksekusi, menunggu penjadwalan CPU

Running: sedang dieksekusi oleh CPU

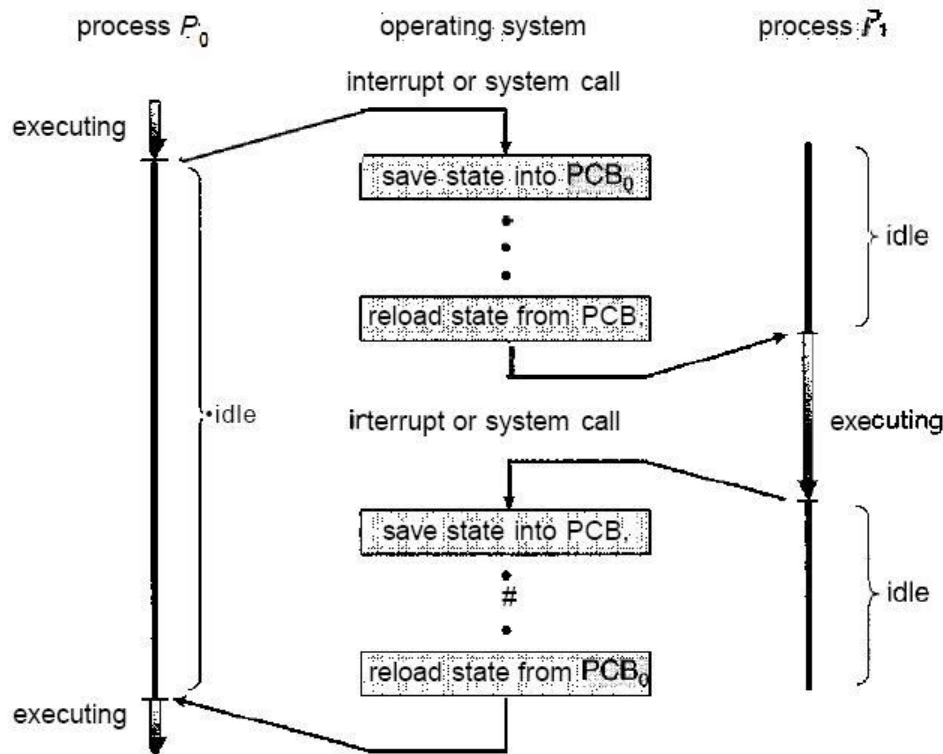
Blocked: sedang di-blocked, e.g. karena menunggu event IO atau karena sengaja di-blocked dgn syscall pause() atau, sleep(), dsb

Pertanyaan: bagaimana dengan process yang baru dibuat dan yang berhenti ?

Dispatcher: modul pada OS yang bertugas untuk switch processor dari process satu ke process yang lain

Context Switching (Alih Konteks)

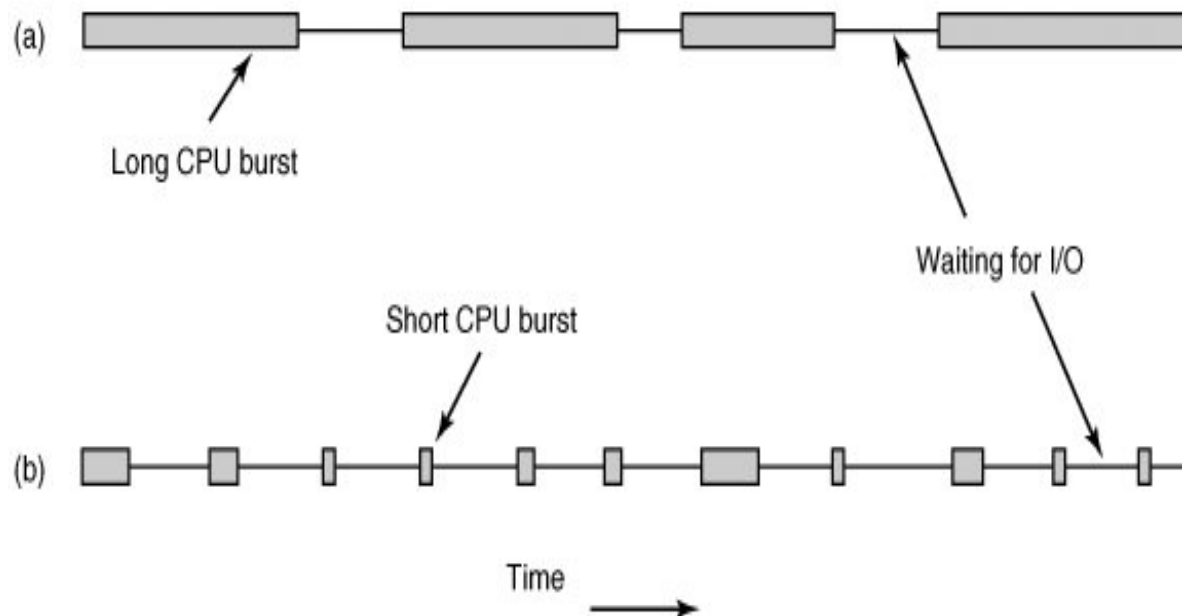
- Perpindahan eksekusi dari process satu ke process yang lain, yang meliputi langkah-langkah sbb:
 - Menghentikan jalannya suatu process dan menyimpan state/context CPU dari process tersebut ke suatu alamat di memory.
 - Menerima context process selanjutnya dari memory dan mengisi kembali CPU register
 - Memulai atau melanjutkan process dengan merujuk ke lokasi yang ditunjuk oleh program counter



- Mengambil sumber daya waktu
→ Extra latency
- Process context switching jauh lebih lambat dibandingkan thread switching

Process Behaviour (Perilaku Proses)

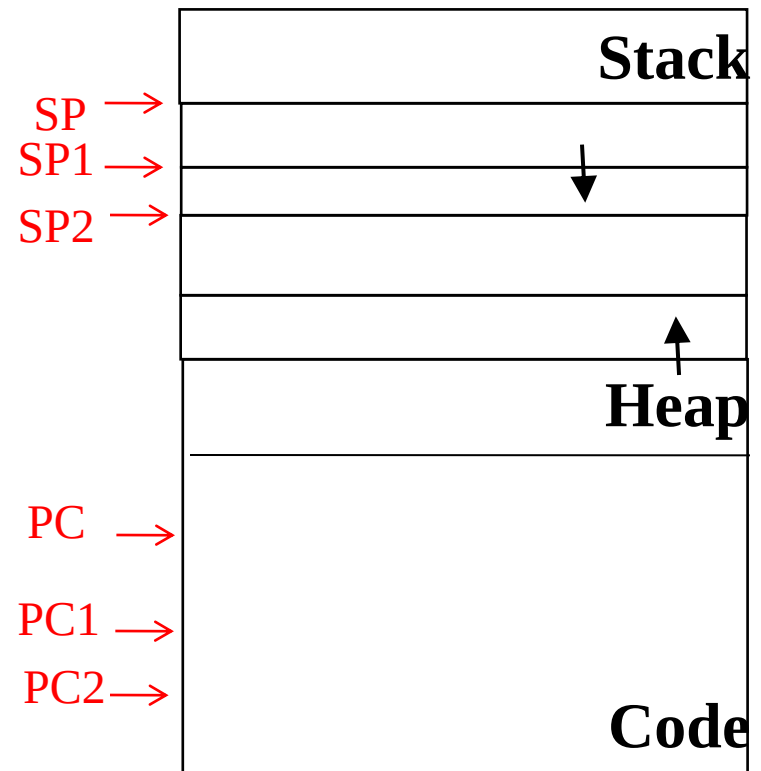
- **CPU-bound:**
 - Penyelesaian pekerjaannya ditentukan sepenuhnya oleh kecepatan CPU. Utilisasi CPU tinggi dan jeda waktu untuk pemrosesan layanan interrupt atau I/O peripherals rendah
- **IO-bound:**
 - Dibatasi oleh kecepatan pemrosesan layanan I/O. Umumnya ditandai oleh short CPU burst yang kemudian disertai waktu menunggu I/O. Utilisasi CPU rendah → multiprocessing dan threads



Threads (1)

- **Pengendali aliran program pada suatu process.**
 - Tiap process memiliki paling tidak satu thread
 - Process yang memiliki beberapa thread (multi-threaded process) memberikan tambahan nuansa concurrency pada tiap process
 - Disebut juga lightweight process, karena memiliki environment yang sama dengan process tapi dengan switching yang lebih sederhana

```
...  
int main() {  
    thrd1=pthread_create();  
    thrd2=pthread_create();  
    ...  
}
```

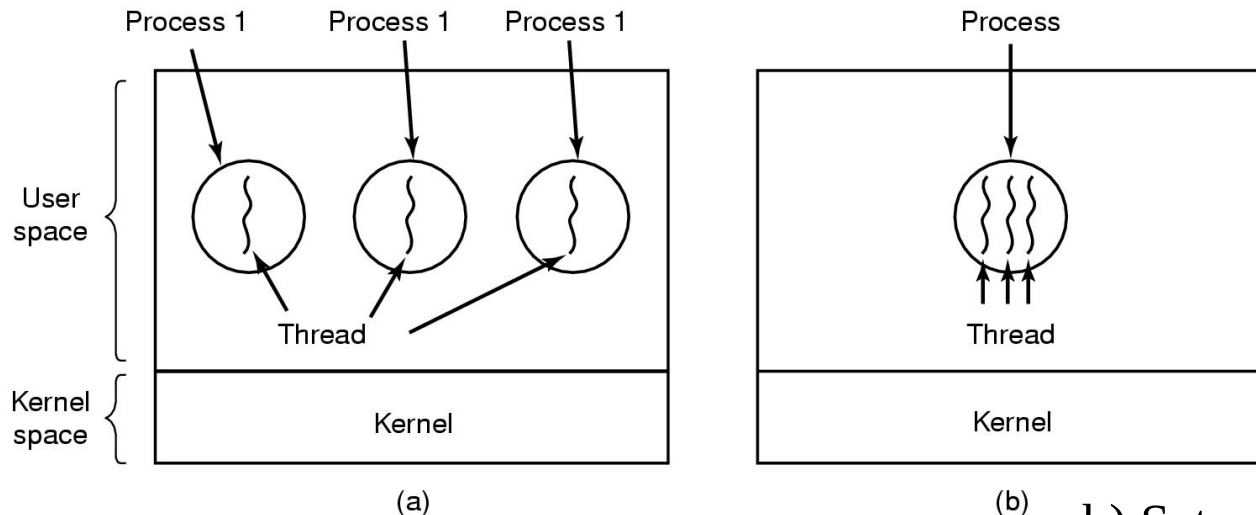


Threads (2)

- Resources (sumber daya) yang dimiliki process dan threads

Per process items	Per thread items
Address space Global variables Open files Child processes Pending alarms Signals and signal handlers Accounting information	Program counter Registers Stack State

- Classical Thread Model



a) Tiga process masing-masing dengan satu thread

b) Satu process dengan tiga threads

Contoh Pemrograman Thread menggunakan pthread library

```
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>
#define NITERS 100000000
int cnt = 0;

void *count(void *arg)
{
    int i;

    for (i=0; i<NITERS; i++)
        cnt++;
}

int main()
{
    pthread_t tid1, tid2;

    pthread_create(&tid1, NULL, count, NULL);
    pthread_create(&tid2, NULL, count, NULL);
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);
    if (cnt != (int)NITERS*2)
        printf("Salah... cnt = %d\n", cnt);
    else
        printf("Benar....cnt=%d\n", cnt);
}
```

Berapa jumlah thread?
Race condition?

Semaphores

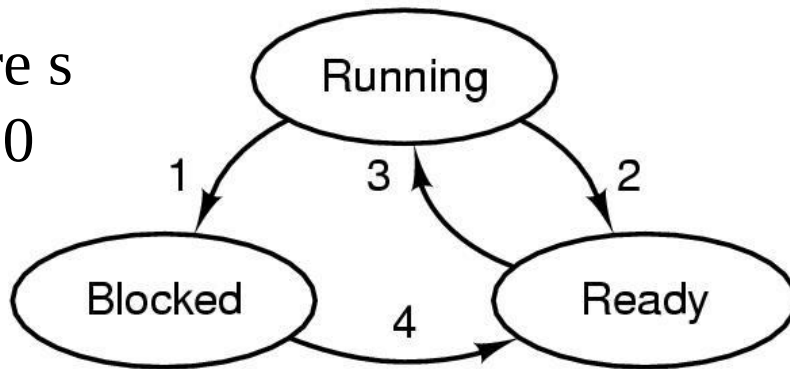
- Variable integer khusus yang dikelola oleh OS (diimplementasikan via system calls) dan digunakan untuk menghitung jumlah blocked/sleeps dan unblocked/wakeups
- Arti nilai semaphore S:
 - Kalau positif → jumlah process yang bisa *decrement* (turunkan satu) S tanpa blocking
 - Kalau negatif → jumlah process yang sedang sleep/blocked dan menunggu untuk dibangunkan/wakeup/un-blocked
 - Kalau nol → tidak ada process yang menunggu, tapi kalau di-*decrement* (turunkan satu), process ini akan di-blocked

Operations on Semaphore S

- **Down(S)**
 $S = S - 1$
If $(S < 0)$ then the calling process is put to sleep/blocked
- **Up(S)**
 $S = S + 1$
If $(S = 0)$ then wakeup/unblock the sleep/blocked process
- Operasi Up() dan Down() bersifat atomic (Atomic Action)
 - Artinya saat operasi dimulai, dia tidak dapat diinterupsi sampai operasi selesai, atau sama sekali tidak dijalankan
 - Operasi: checking dan updating nilai semaphore, dan action (block atau unblock)
- Notasi yang lain menurut karya tulis dari Dijkstra (pengusul konsep semaphore):
 $P() = \text{Down}()$ $V() = \text{Up}()$

Three-State Process Model

Down
semaphore s
when $s = 0$



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

Blocked

Up semaphore s , to change
the state to Ready

Arti nilai semaphore:

Kalau positif, artinya nilai tersebut menunjukkan jumlah process yang bisa decrement S tanpa blocking

Kalau negatif, artinya nilai tersebut menunjukkan jumlah process yang sedang sleep/blocked dan menunggu untuk dibangunkan/wakeup/unblocked

Mutex Variable menggunakan Binary Semaphore

- **Mutex variable → 2 states yaitu unlocked dan locked**
 - Untuk melindungi critical region
- **Binary semaphore → hanya ada 2 kemungkinan nilai yaitu 0 dan 1**
- **Inisialisasi semaphore ke nilai 1**
- **Agar hanya ada satu process berada di critical region yang dilindungi oleh semaphore, maka lakukan**
 - **Down()** untuk masuk critical region
 - **Up()** untuk keluar dari critical region

- **Contoh:**

```
semaphore mutex = 1;
```

```
down(&mutex);
```

```
/* Critical
```

```
Section */
```

```
up(&mutex);
```

Producer-Consumer Problem menggunakan semaphores

```
sem_init(&empty, 0, SIZE);  
sem_init(&full, 0, 0);  
sem_init(&mutex, 0, 1);
```



Inisialisasi
semaphore untuk
multi-thread
application

```
#define N 100  
typedef int semaphore;  
semaphore mutex = 1;  
semaphore empty = N;  
semaphore full = 0;
```

Event semaphores
Hitung jumlah slot kosong
Hitung jumlah slot yang terisi

```
void producer(void)  
{  
    int item;  
  
    while (TRUE) {  
        item = produce_item();  
        down(&empty);  
        down(&mutex);  
        insert_item(item);  
        up(&mutex);  
        up(&full);  
    }  
}
```

```
void consumer(void)  
{  
    int item;  
  
    while (TRUE) {  
        down(&full);  
        down(&mutex);  
        item = remove_item();  
        up(&mutex);  
        up(&empty);  
        consume_item(item);  
    }  
}
```

Akhir Kuliah Minggu 7

Terima kasih atas perhatiannya!